My Project

Generated by Doxygen 1.8.17

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 ressources_s Struct Reference . . . 5

3.1.1 Detailed Description . . . 5

3.1.2 Member Data Documentation . . . 5

3.1.2.1 background . . . 5

3.1.2.2 font . . . 6

3.1.2.3 ligne_arrivee . . . 6

3.1.2.4 meteorite . . . 6

3.1.2.5 missile . . . 6

3.1.2.6 vaisseau . . . 6

3.2 sprite_s Struct Reference . . . 6

3.2.1 Detailed Description . . . 7

3.2.2 Member Data Documentation . . . 7

3.2.2.1 h . . . 7

3.2.2.2 w . . . 7

3.2.2.3 x . . . 7

3.2.2.4 y . . . 7

3.3 world_s Struct Reference . . . 8

3.3.1 Detailed Description . . . 8

3.3.2 Member Data Documentation . . . 8

3.3.2.1 gameover . . . 8

3.3.2.2 ligne_arrivee . . . 9

3.3.2.3 missiles . . . 9

3.3.2.4 time . . . 9

3.3.2.5 vaisseau . . . 9

3.3.2.6 vy . . . 9

3.3.2.7 walls . . . 9

3.3.2.8 win . . . 9

4 File Documentation 11

4.1 affichage.c File Reference . . . 11

4.1.1 Detailed Description . . . 12

4.1.2 Function Documentation . . . 12

4.1.2.1 apply_background() . . . 12

4.1.2.2 apply_sprite() . . . 12

4.1.2.3 apply_texts() . . . 13

4.1.2.4 apply_walls() . . . 13

4.1.2.5 clean_ressources() . . . 14
4.1.2.6 init_textures() . . . 14
4.1.2.7 refresh_graphics() . . . 14
4.2 affichage.h File Reference . . . 14
4.2.1 Detailed Description . . . 16
4.2.2 Function Documentation . . . 16
4.2.2.1 apply_background() . . . 16
4.2.2.2 apply_sprite() . . . 17
4.2.2.3 apply_texts() . . . 17
4.2.2.4 apply_walls() . . . 17
4.2.2.5 clean_ressources() . . . 18
4.2.2.6 init_textures() . . . 18
4.2.2.7 refresh_graphics() . . . 18
4.3 constantes.h File Reference . . . 19
4.3.1 Detailed Description . . . 19
4.4 logique.c File Reference . . . 20
4.4.1 Detailed Description . . . 21
4.4.2 Function Documentation . . . 21
4.4.2.1 handle_sprites_collision() . . . 21
4.4.2.2 init_data() . . . 21
4.4.2.3 init_sprite() . . . 23
4.4.2.4 init_walls() . . . 23
4.4.2.5 is_finished() . . . 23
4.4.2.6 shoot_missile() . . . 24
4.4.2.7 sprites_collide() . . . 24
4.4.2.8 update_data() . . . 24
4.4.2.9 update_finishline() . . . 25
4.4.2.10 update_missiles() . . . 25
4.4.2.11 update_time() . . . 25
4.4.2.12 update_walls() . . . 26
4.5 logique.h File Reference . . . 26
4.5.1 Detailed Description . . . 28
4.5.2 Function Documentation . . . 28
4.5.2.1 handle_sprites_collision() . . . 28
4.5.2.2 init_data() . . . 29
4.5.2.3 init_sprite() . . . 29
4.5.2.4 init_walls() . . . 29
4.5.2.5 is_finished() . . . 30
4.5.2.6 shoot_missile() . . . 30
4.5.2.7 sprites_collide() . . . 30
4.5.2.8 update_data() . . . 30
4.5.2.9 update_finishline() . . . 31

4.5.2.10 update_missiles() . . . 31
4.5.2.11 update_time() . . . 31
4.5.2.12 update_walls() . . . 32
4.6 main.c File Reference . . . 32
4.6.1 Detailed Description . . . 33
4.6.2 Function Documentation . . . 33
4.6.2.1 clean() . . . 33
4.6.2.2 handle_events() . . . 33
4.6.2.3 init() . . . 34
4.7 sdl2-light.c File Reference . . . 34
4.7.1 Detailed Description . . . 35
4.7.2 Function Documentation . . . 36
4.7.2.1 apply_texture() . . . 36
4.7.2.2 clean_sdl() . . . 36
4.7.2.3 clean_texture() . . . 36
4.7.2.4 clear_renderer() . . . 37
4.7.2.5 init_sdl() . . . 37
4.7.2.6 load_image() . . . 37
4.7.2.7 pause() . . . 38
4.7.2.8 update_screen() . . . 38
4.8 sdl2-light.h File Reference . . . 38
4.8.1 Detailed Description . . . 40
4.8.2 Function Documentation . . . 40
4.8.2.1 apply_texture() . . . 40
4.8.2.2 clean_sdl() . . . 41
4.8.2.3 clean_texture() . . . 41
4.8.2.4 clear_renderer() . . . 41
4.8.2.5 init_sdl() . . . 41
4.8.2.6 load_image() . . . 42
4.8.2.7 pause() . . . 42
4.8.2.8 update_screen() . . . 43

Index **45**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ressources_s
: Représentation pour stocker les ressources nécessaires à l'affichage graphique 5

sprite_s
: Type qui correspond au sprite 6

world_s
: Représentation du monde du jeu 8

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

affichage.c
: Contient l'ensemble des fonctions relatives à l'affichage des éléments du jeu 11

affichage.h
: Header de affichage.c 14

constantes.h
: Contient l'ensemble des constantes du jeu 19

logique.c
: Contient l'ensemble des fonctions relatives à la logique du jeu 20

logique.h
: Header de logique.c 26

main.c
: Programme principal du jeu 32

sdl2-light.c
: Sur-couche de SDL2 pour simplifier son utilisation pour le projet 34

sdl2-light.h
: En-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet 38

sdl2-ttf-light.h . . . . . **??**

Chapter 3

Class Documentation

3.1 ressources_s Struct Reference

Représentation pour stocker les ressources nécessaires à l'affichage graphique.

`#include <affichage.h>`

Public Attributes

- SDL_Texture * background
- SDL_Texture * vaisseau
- SDL_Texture * ligne_arrivee
- SDL_Texture * meteorite
- SDL_Texture * missile
- TTF_Font * font

3.1.1 Detailed Description

Représentation pour stocker les ressources nécessaires à l'affichage graphique.

3.1.2 Member Data Documentation

3.1.2.1 background

`SDL_Texture* ressources_s::background`

Texture liée à l'image du fond de l'écran.

3.1.2.2 font

```
TTF_Font* ressources_s::font
```

Texture liée à la police du texte

3.1.2.3 ligne_arrivee

```
SDL_Texture* ressources_s::ligne_arrivee
```

Texture liée à l'image de la ligne d'arrivée

3.1.2.4 meteorite

```
SDL_Texture* ressources_s::meteorite
```

Texture liée à l'image d'une météorite

3.1.2.5 missile

```
SDL_Texture* ressources_s::missile
```

Texture liée à l'image d'une missile

3.1.2.6 vaisseau

```
SDL_Texture* ressources_s::vaisseau
```

Texture liée à l'image du vaisseau.

The documentation for this struct was generated from the following file:

- affichage.h

3.2 sprite_s Struct Reference

Type qui correspond au sprite.

```
#include <logique.h>
```

Public Attributes

- float x
- float y
- float w
- float h

3.2.1 Detailed Description

Type qui correspond au sprite.

3.2.2 Member Data Documentation

3.2.2.1 h

```
float sprite_s::h
```

Champ indiquant la hauteur h du sprite

3.2.2.2 w

```
float sprite_s::w
```

Champ indiquant la largeur w du sprite

3.2.2.3 x

```
float sprite_s::x
```

Champ indiquant la position x du sprite

3.2.2.4 y

```
float sprite_s::y
```

Champ indiquant la position y du sprite

The documentation for this struct was generated from the following file:

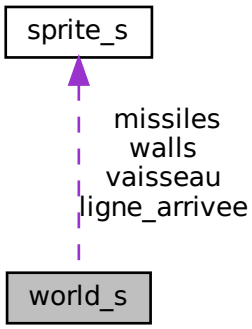
- logique.h

3.3 world_s Struct Reference

Représentation du monde du jeu.

`#include <logique.h>`

Collaboration diagram for world_s:



Public Attributes

- sprite_t vaisseau
- sprite_t ligne_arrivee
- sprite_t walls [6]
- sprite_t missiles [10]
- float vy
- int gameover
- int win
- float time

3.3.1 Detailed Description

Représentation du monde du jeu.

3.3.2 Member Data Documentation

3.3.2.1 gameover

`int world_s::gameover`

Champ indiquant si on a perdu

3.3.2.2 ligne_arrivee

```
sprite_t world_s::ligne_arrivee
```

Champ représentant le sprite de la ligne d'arrivée

3.3.2.3 missiles

```
sprite_t world_s::missiles[10]
```

Champ représentant le tableau des missiles tirés par le vaisseau

3.3.2.4 time

```
float world_s::time
```

Champ représentant le temps écoulé depuis le début du jeu

3.3.2.5 vaisseau

```
sprite_t world_s::vaisseau
```

Champ représentant le sprite du vaisseau

3.3.2.6 vy

```
float world_s::vy
```

Champ représentant la vitesse de déplacement de la ligne d'arrivée

3.3.2.7 walls

```
sprite_t world_s::walls[6]
```

Champ représentant le tableau du sprite des murs de météorites

3.3.2.8 win

```
int world_s::win
```

Champ indiquant si on a gagné

The documentation for this struct was generated from the following file:

- logique.h

Chapter 4

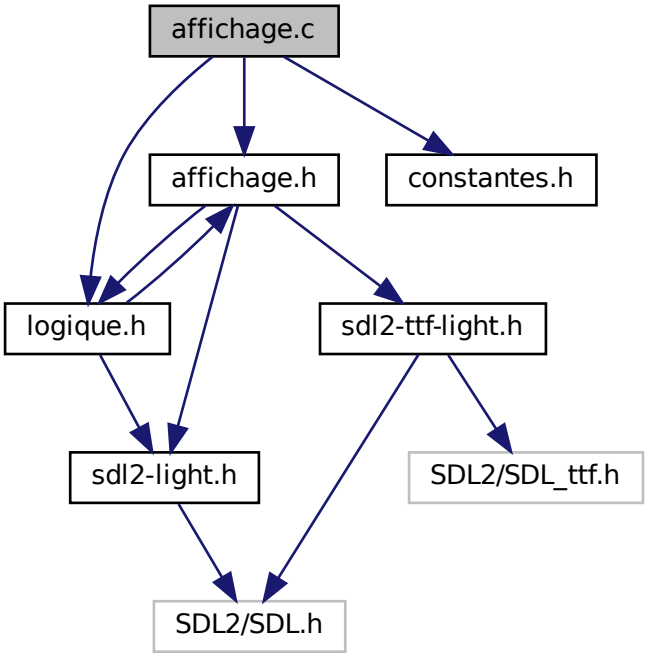
File Documentation

4.1 affichage.c File Reference

Contient l'ensemble des fonctions relatives à l'affichage des éléments du jeu.

```
#include "affichage.h"
#include "logique.h"
#include "constantes.h"
```

Include dependency graph for affichage.c:



Functions

- void init_textures (SDL_Renderer ∗renderer, ressources_t ∗ressources)

  La fonction initialise les texures.
- void apply_walls (ressources_t ∗ressources, SDL_Renderer ∗renderer, world_t ∗world)

  fonction qui applique la texture des murs de météorites
- void apply_sprite (SDL_Renderer ∗renderer, ressources_t ∗ressources, world_t ∗world)

  La fonction applique la texture du sprite sur le renderer lié à l'écran de jeu.
- void apply_background (SDL_Renderer ∗renderer, ressources_t ∗ressources)

  La fonction applique la texture du fond sur le renderer lié à l'écran de jeu.
- void apply_texts (SDL_Renderer ∗renderer, world_t ∗world, ressources_t ∗ressources)

  La fonction affiche le timer et les messages de fin sur le renderer lié à l'écran de jeu.
- void refresh_graphics (SDL_Renderer ∗renderer, world_t ∗world, ressources_t ∗ressources)

  La fonction rafraichit l'écran en fonction de l'état des données du monde.
- void clean_ressources (ressources_t ∗ressources)

  La fonction nettoie les ressources.

4.1.1 Detailed Description

Contient l'ensemble des fonctions relatives à l'affichage des éléments du jeu.

Author

Valentini Andréa & Viale Romain

4.1.2 Function Documentation

4.1.2.1 apply_background()

```
void apply_background (
            SDL_Renderer * renderer,
            ressources_t * ressources )
```

La fonction applique la texture du fond sur le renderer lié à l'écran de jeu.

Parameters

renderer	le renderer
ressources	les ressources du jeu

4.1.2.2 apply_sprite()

```
void apply_sprite (
            SDL_Renderer * renderer,
```

```
            ressources_t * ressources,
            world_t * world )
```

La fonction applique la texture du sprite sur le renderer lié à l'écran de jeu.

Parameters

renderer	le renderer
ressources	les ressources du jeu
world	le monde du jeu

4.1.2.3 apply_texts()

```
void apply_texts (
            SDL_Renderer * renderer,
            world_t * world,
            ressources_t * ressources )
```

La fonction affiche le timer et les messages de fin sur le renderer lié à l'écran de jeu.

Parameters

renderer	le renderer
world	le monde du jeu
ressources	les ressources du jeu

4.1.2.4 apply_walls()

```
void apply_walls (
            ressources_t * ressources,
            SDL_Renderer * renderer,
            world_t * world )
```

fonction qui applique la texture des murs de météorites

Parameters

ressources	les ressources du jeu
renderer	le renderer
world	le monde du jeu

4.1.2.5 clean_ressources()

```
void clean_ressources (
            ressources_t * ressources )
```

La fonction nettoie les ressources.

Parameters

ressources	les ressources

4.1.2.6 init_textures()

```
void init_textures (
            SDL_Renderer * renderer,
            ressources_t * ressources )
```

La fonction initialise les texures.

Parameters

screen	la surface correspondant à l'écran de jeu
ressources	les ressources du jeu

4.1.2.7 refresh_graphics()

```
void refresh_graphics (
            SDL_Renderer * renderer,
            world_t * world,
            ressources_t * ressources )
```

La fonction rafraichit l'écran en fonction de l'état des données du monde.

Parameters

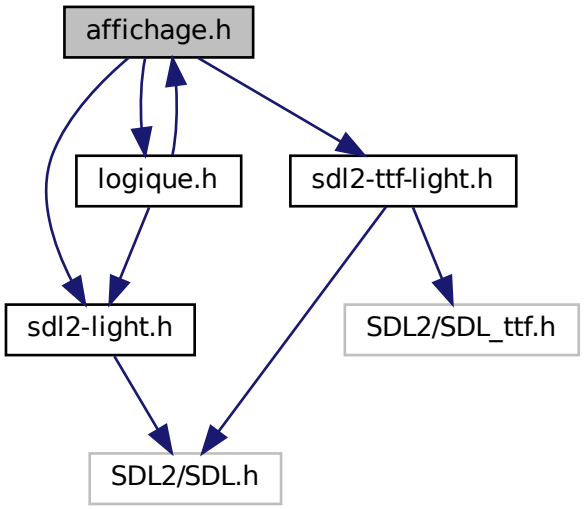
renderer	le renderer
world	le monde du jeu
ressources	les ressources du jeu

4.2 affichage.h File Reference

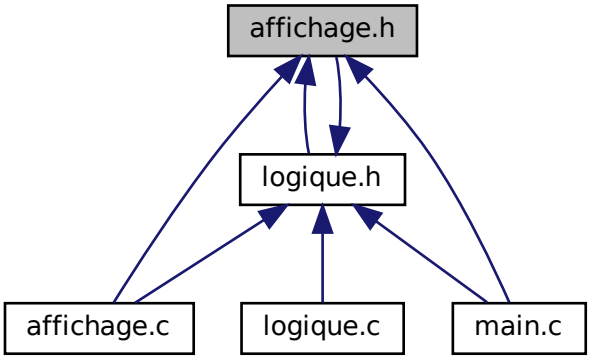
Header de affichage.c.

```
#include "sdl2-light.h"
#include "sdl2-ttf-light.h"
#include "logique.h"
```

Include dependency graph for affichage.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct ressources_s

  Représentation pour stocker les ressources nécessaires à l'affichage graphique.

Typedefs

- typedef struct ressources_s ressources_t

  Représentation pour stocker les ressources nécessaires à l'affichage graphique.

Functions

- void init_textures (SDL_Renderer ∗renderer, ressources_t ∗ressources)

  La fonction initialise les texures.
- void apply_walls (ressources_t ∗ressources, SDL_Renderer ∗renderer, world_t ∗world)

  fonction qui applique la texture des murs de météorites
- void apply_sprite (SDL_Renderer ∗renderer, ressources_t ∗ressources, world_t ∗world)

  La fonction applique la texture du sprite sur le renderer lié à l'écran de jeu.
- void apply_background (SDL_Renderer ∗renderer, ressources_t ∗ressources)

  La fonction applique la texture du fond sur le renderer lié à l'écran de jeu.
- void apply_texts (SDL_Renderer ∗renderer, world_t ∗world, ressources_t ∗ressources)

  La fonction affiche le timer et les messages de fin sur le renderer lié à l'écran de jeu.
- void refresh_graphics (SDL_Renderer ∗renderer, world_t ∗world, ressources_t ∗ressources)

  La fonction rafraichit l'écran en fonction de l'état des données du monde.
- void clean_ressources (ressources_t ∗ressources)

  La fonction nettoie les ressources.

4.2.1 Detailed Description

Header de affichage.c.

Author

Valentini Andréa & Viale Romain

4.2.2 Function Documentation

4.2.2.1 apply_background()

```
void apply_background (
            SDL_Renderer * renderer,
            ressources_t * ressources )
```

La fonction applique la texture du fond sur le renderer lié à l'écran de jeu.

Parameters

renderer	le renderer
ressources	les ressources du jeu

4.2.2.2 apply_sprite()

```
void apply_sprite (
            SDL_Renderer * renderer,
            ressources_t * ressources,
            world_t * world )
```

La fonction applique la texture du sprite sur le renderer lié à l'écran de jeu.

Parameters

renderer	le renderer
ressources	les ressources du jeu
world	le monde du jeu

4.2.2.3 apply_texts()

```
void apply_texts (
            SDL_Renderer * renderer,
            world_t * world,
            ressources_t * ressources )
```

La fonction affiche le timer et les messages de fin sur le renderer lié à l'écran de jeu.

Parameters

renderer	le renderer
world	le monde du jeu
ressources	les ressources du jeu

4.2.2.4 apply_walls()

```
void apply_walls (
            ressources_t * ressources,
            SDL_Renderer * renderer,
            world_t * world )
```

fonction qui applique la texture des murs de météorites

Parameters

ressources	les ressources du jeu
renderer	le renderer
world	le monde du jeu

4.2.2.5 clean_ressources()

```
void clean_ressources (
            ressources_t * ressources )
```

La fonction nettoie les ressources.

Parameters

ressources	les ressources

4.2.2.6 init_textures()

```
void init_textures (
            SDL_Renderer * renderer,
            ressources_t * ressources )
```

La fonction initialise les texures.

Parameters

screen	la surface correspondant à l'écran de jeu
ressources	les ressources du jeu

4.2.2.7 refresh_graphics()

```
void refresh_graphics (
            SDL_Renderer * renderer,
            world_t * world,
            ressources_t * ressources )
```

La fonction rafraichit l'écran en fonction de l'état des données du monde.

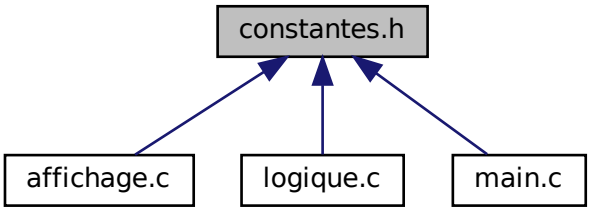
Parameters

renderer	le renderer
world	le monde du jeu
ressources	les ressources du jeu

4.3 constantes.h File Reference

Contient l'ensemble des constantes du jeu.

This graph shows which files directly or indirectly include this file:



Macros

- #define SCREEN_WIDTH 300
  Largeur de l'écran de jeu.
- #define SCREEN_HEIGHT 480
  Hauteur de l'écran de jeu.
- #define SHIP_SIZE 32
  Taille d'un vaisseau.
- #define METEORITE_SIZE 32
  Taille d'un météorite.
- #define MISSILE_WIDTH 9
  Largeur d'un missile.
- #define MISSILE_HEIGHT 17
  Hauteur d'un missile.
- #define FINISH_LINE_HEIGHT -960
  Hauteur de la ligne d'arrivée.
- #define MOVING_STEP 10
  Pas de déplacement horizontal du vaisseau.
- #define MISSILE_SPEED 2
  Vitesse d'un missile.
- #define MISSILE_COOLDOWN 300
  Cooldown d'un missile (en ms)
- #define INITIAL_SPEED 1
  Vitesse initiale de déplacement vertical des éléments du jeu.

4.3.1 Detailed Description

Contient l'ensemble des constantes du jeu.

Author

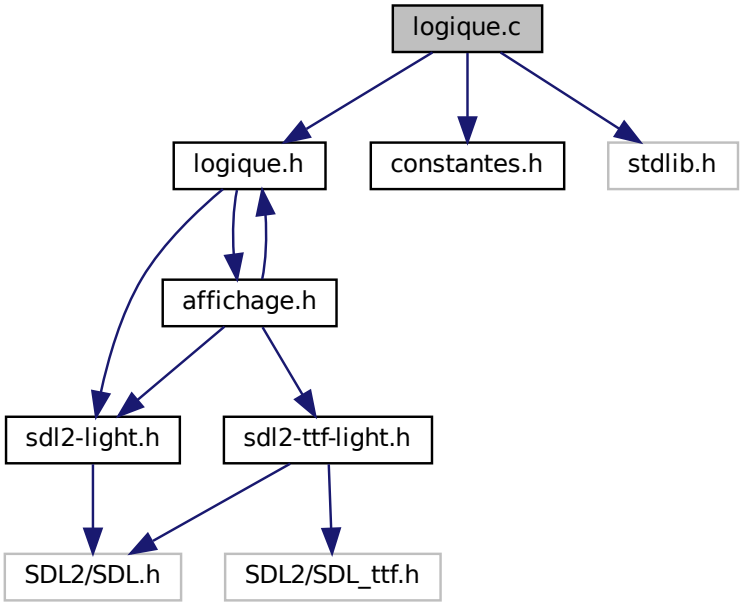
Valentini Andréa & Viale Romain

4.4 logique.c File Reference

Contient l'ensemble des fonctions relatives à la logique du jeu.

```
#include "logique.h"
#include "constantes.h"
#include <stdlib.h>
```

Include dependency graph for logique.c:



Functions

- void init_sprite (sprite_t *sprite, int x, int y, int w, int h)

  La fonction initialise la position du sprite.
- void init_walls (world_t *world)

  La fonction initialise la position des murs de météorites.
- void init_data (world_t *world)

  La fonction initialise les données du monde du jeu.
- int sprites_collide (sprite_t *sp1, sprite_t *sp2)

  La fonction renvoie l'état de collision de 2 sprites (0 ou 1)
- void handle_sprites_collision (world_t *world, sprite_t *sp1, sprite_t *sp2, int make_disappear, int make_win)

  La fonction gère les collisions entre deux sprites.
- int is_finished (world_t *world)

  Renvoie l'état de fin du jeu (jeu gagné/jeu perdu)
- void update_time (world_t *world, unsigned int currentTime, unsigned int *lastTimeCheck)

  Met à jour le temps écoulé
- void update_finishline (world_t *world)

  Met à jour la position de la ligne d'arrivée + gère les collisions.

- void update_walls (world_t *world)

  Met à jour la position des murs de météorites + gère les collisions.
- void shoot_missile (world_t *world)

  Tire un missile à partir du vaisseau.
- void update_missiles (world_t *world)

  Met à jour la position du missile + gère les collisions.
- void update_data (world_t *world, ressources_t *ressources)

  La fonction met à jour les données en tenant compte de la physique du monde.

4.4.1 Detailed Description

Contient l'ensemble des fonctions relatives à la logique du jeu.

Author

Valentini Andréa & Viale Romain

4.4.2 Function Documentation

4.4.2.1 handle_sprites_collision()

```
void handle_sprites_collision (
            world_t * world,
            sprite_t * sp1,
            sprite_t * sp2,
            int make_disappear,
            int make_win )
```

La fonction gère les collisions entre deux sprites.

Parameters

world	les données du monde
sp1	le premier sprite
sp2	le deuxième sprite
make_disappear	1 si sp1 doit disparaitre en cas de collision, 0 sinon
make_win	1 si la collision déclenche la victoire, 0 si défaite, tout autre valeur si ca ne doit rien faire
ressources	les ressources du jeu

4.4.2.2 init_data()

```
void init_data (
            world_t * world )
```

La fonction initialise les données du monde du jeu.

Parameters

world	les données du monde

4.4.2.3 init_sprite()

```
void init_sprite (
            sprite_t * sprite,
            int x,
            int y,
            int w,
            int h )
```

La fonction initialise la position du sprite.

Parameters

sprite	le sprite du jeu
x	la position x du sprite
y	la position y du sprite
w	la largeur du sprite
h	la hauteur du sprite

4.4.2.4 init_walls()

```
void init_walls (
            world_t * world )
```

La fonction initialise la position des murs de météorites.

Parameters

world	les données du monde

4.4.2.5 is_finished()

```
int is_finished (
            world_t * world )
```

Renvoie l'état de fin du jeu (jeu gagné/jeu perdu)

Parameters

world	les données du monde

4.4.2.6 shoot_missile()

```
void shoot_missile (
            world_t * world )
```

Tire un missile à partir du vaisseau.

Parameters

world	les données du monde

4.4.2.7 sprites_collide()

```
int sprites_collide (
            sprite_t * sp1,
            sprite_t * sp2 )
```

La fonction renvoie l'état de collision de 2 sprites (0 ou 1)

Parameters

sp1	le premier sprite
sp2	le deuxième sprite

4.4.2.8 update_data()

```
void update_data (
            world_t * world,
            ressources_t * ressources )
```

La fonction met à jour les données en tenant compte de la physique du monde.

Parameters

world	les données du monde
ressources	les ressources du jeu

4.4.2.9 update_finishline()

```
void update_finishline (
            world_t * world )
```

Met à jour la position de la ligne d'arrivée + gère les collisions.

Parameters

world	les données du monde
currentTime	le temps écoulé actuel
lastTimeCheck	le temps écoulé au dernier check

4.4.2.10 update_missiles()

```
void update_missiles (
            world_t * world )
```

Met à jour la position du missile + gère les collisions.

Parameters

world	les données du monde

4.4.2.11 update_time()

```
void update_time (
            world_t * world,
            unsigned int currentTime,
            unsigned int * lastTimeCheck )
```

Met à jour le temps écoulé

Parameters

world	les données du monde
currentTime	le temps écoulé actuel
lastTimeCheck	le temps écoulé au dernier check

4.4.2.12 update_walls()

```
void update_walls (
            world_t * world )
```

Met à jour la position des murs de météorites + gère les collisions.

Parameters

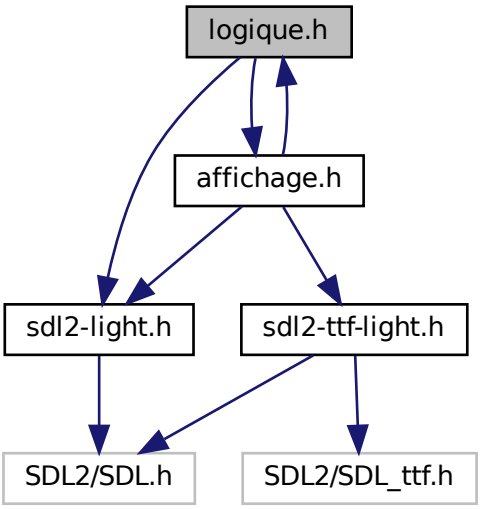
world	les données du monde
ressources	les ressources du jeu

4.5 logique.h File Reference

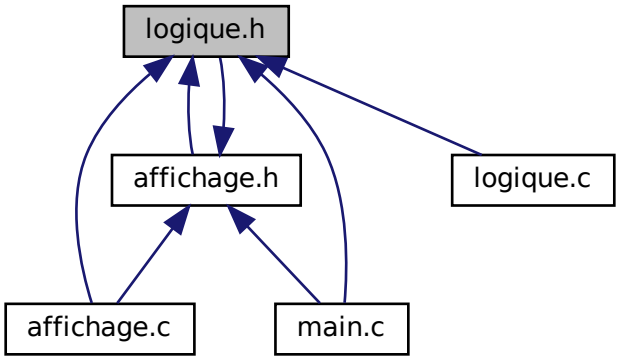
Header de logique.c.

```
#include "sdl2-light.h"
#include "affichage.h"
```

Include dependency graph for logique.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct sprite_s

  Type qui correspond au sprite.
- struct world_s

  Représentation du monde du jeu.

Typedefs

- typedef struct sprite_s sprite_t

  Type qui correspond au sprite.
- typedef struct world_s world_t

  Représentation du monde du jeu.

Functions

- void init_sprite (sprite_t *sprite, int x, int y, int w, int h)

  La fonction initialise la position du sprite.
- void init_walls (world_t *world)

  La fonction initialise la position des murs de météorites.
- void init_data (world_t *world)

  La fonction initialise les données du monde du jeu.
- int sprites_collide (sprite_t *sp1, sprite_t *sp2)

  La fonction renvoie l'état de collision de 2 sprites (0 ou 1)
- void handle_sprites_collision (world_t *world, sprite_t *sp1, sprite_t *sp2, int make_disappear, int make_win)

  La fonction gère les collisions entre deux sprites.
- int is_finished (world_t *world)

  Renvoie l'état de fin du jeu (jeu gagné/jeu perdu)
- void update_time (world_t *world, unsigned int currentTime, unsigned int *lastTimeCheck)

Met à jour le temps écoulé

- void update_finishline (world_t *world)

  Met à jour la position de la ligne d'arrivée + gère les collisions.
- void update_walls (world_t *world)

  Met à jour la position des murs de météorites + gère les collisions.
- void shoot_missile (world_t *world)

  Tire un missile à partir du vaisseau.
- void update_missiles (world_t *world)

  Met à jour la position du missile + gère les collisions.
- void update_data (world_t *world, ressources_t *ressources)

  La fonction met à jour les données en tenant compte de la physique du monde.

4.5.1 Detailed Description

Header de logique.c.

Author

Valentini Andréa & Viale Romain

4.5.2 Function Documentation

4.5.2.1 handle_sprites_collision()

```
void handle_sprites_collision (
            world_t * world,
            sprite_t * sp1,
            sprite_t * sp2,
            int make_disappear,
            int make_win )
```

La fonction gère les collisions entre deux sprites.

Parameters

world	les données du monde
sp1	le premier sprite
sp2	le deuxième sprite
make_disappear	1 si sp1 doit disparaitre en cas de collision, 0 sinon
make_win	1 si la collision déclenche la victoire, 0 si défaite, tout autre valeur si ca ne doit rien faire
ressources	les ressources du jeu

4.5.2.2 init_data()

```
void init_data (
            world_t * world )
```

La fonction initialise les données du monde du jeu.

Parameters

world	les données du monde

4.5.2.3 init_sprite()

```
void init_sprite (
            sprite_t * sprite,
            int x,
            int y,
            int w,
            int h )
```

La fonction initialise la position du sprite.

Parameters

sprite	le sprite du jeu
x	la position x du sprite
y	la position y du sprite
w	la largeur du sprite
h	la hauteur du sprite

4.5.2.4 init_walls()

```
void init_walls (
            world_t * world )
```

La fonction initialise la position des murs de météorites.

Parameters

world	les données du monde

4.5.2.5 is_finished()

```
int is_finished (
            world_t * world )
```

Renvoie l'état de fin du jeu (jeu gagné/jeu perdu)

Parameters

world	les données du monde

4.5.2.6 shoot_missile()

```
void shoot_missile (
            world_t * world )
```

Tire un missile à partir du vaisseau.

Parameters

world	les données du monde

4.5.2.7 sprites_collide()

```
int sprites_collide (
            sprite_t * sp1,
            sprite_t * sp2 )
```

La fonction renvoie l'état de collision de 2 sprites (0 ou 1)

Parameters

sp1	le premier sprite
sp2	le deuxième sprite

4.5.2.8 update_data()

```
void update_data (
            world_t * world,
            ressources_t * ressources )
```

La fonction met à jour les données en tenant compte de la physique du monde.

Parameters

world	les données du monde
ressources	les ressources du jeu

4.5.2.9 update_finishline()

```
void update_finishline (
            world_t * world )
```

Met à jour la position de la ligne d'arrivée + gère les collisions.

Parameters

world	les données du monde
currentTime	le temps écoulé actuel
lastTimeCheck	le temps écoulé au dernier check

4.5.2.10 update_missiles()

```
void update_missiles (
            world_t * world )
```

Met à jour la position du missile + gère les collisions.

Parameters

world	les données du monde

4.5.2.11 update_time()

```
void update_time (
            world_t * world,
            unsigned int currentTime,
            unsigned int * lastTimeCheck )
```

Met à jour le temps écoulé

Parameters

world	les données du monde
currentTime	le temps écoulé actuel
lastTimeCheck	le temps écoulé au dernier check

4.5.2.12 update_walls()

```
void update_walls (
            world_t * world )
```

Met à jour la position des murs de météorites + gère les collisions.

Parameters

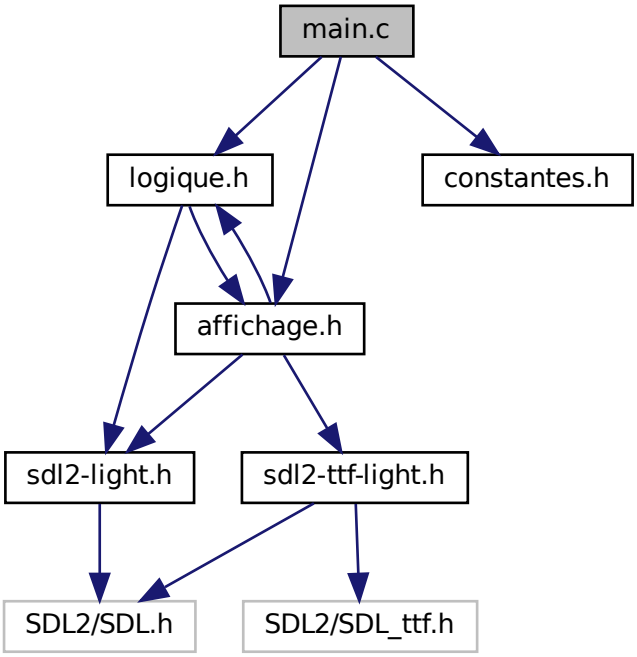
world	les données du monde
ressources	les ressources du jeu

4.6 main.c File Reference

Programme principal du jeu.

```
#include "logique.h"
#include "affichage.h"
#include "constantes.h"
```

Include dependency graph for main.c:



Functions

- void init (SDL_Window **window, SDL_Renderer **renderer, ressources_t *ressources, world_t *world)

  fonction qui initialise le jeu: initialisation de la partie graphique (SDL), chargement des ressources, initialisation des données
- void handle_events (SDL_Event *event, world_t *world)

  fonction qui gère les évènements ayant eu lieu et qui n'ont pas encore été traités
- void clean (SDL_Window *window, SDL_Renderer *renderer, ressources_t *ressources, world_t *world)

  fonction qui nettoie le jeu: nettoyage de la partie graphique (SDL), nettoyage des ressources, nettoyage des données
- void print_sprite (sprite_t *sprite)

  Fonction de test des coordonnées du sprite.
- int main (int argc, char *args[])

  programme principal qui implémente la boucle du jeu

4.6.1 Detailed Description

Programme principal du jeu.

Author

Valentini Andréa & Viale Romain

4.6.2 Function Documentation

4.6.2.1 clean()

```
void clean (
            SDL_Window * window,
            SDL_Renderer * renderer,
            ressources_t * ressources,
            world_t * world )
```

fonction qui nettoie le jeu: nettoyage de la partie graphique (SDL), nettoyage des ressources, nettoyage des données

Parameters

window	la fenêtre du jeu
renderer	le renderer
ressources	les ressources
world	le monde

4.6.2.2 handle_events()

```
void handle_events (
```

```
            SDL_Event * event,
            world_t * world )
```

fonction qui gère les évènements ayant eu lieu et qui n'ont pas encore été traités

Parameters

event	paramètre qui contient les événements
world	les données du monde

4.6.2.3 init()

```
void init (
            SDL_Window ** window,
            SDL_Renderer ** renderer,
            ressources_t * ressources,
            world_t * world )
```

fonction qui initialise le jeu: initialisation de la partie graphique (SDL), chargement des ressources, initialisation des données

Parameters

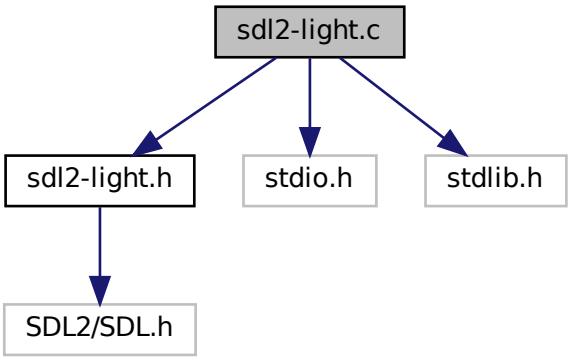
window	la fenêtre du jeu
renderer	le renderer
ressources	les ressources du jeu
world	le monde du jeu

4.7 sdl2-light.c File Reference

sur-couche de SDL2 pour simplifier son utilisation pour le projet

```
#include "sdl2-light.h"
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for sdl2-light.c:



Functions

- int init_sdl (SDL_Window **window, SDL_Renderer **renderer, int width, int height)

  La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.
- SDL_Texture * load_image (const char path[ ], SDL_Renderer *renderer)

  La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.
- void apply_texture (SDL_Texture *texture, SDL_Renderer *renderer, int x, int y)

  La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.
- void clean_texture (SDL_Texture *texture)

  La fonction nettoie une texture en mémoire.
- void clear_renderer (SDL_Renderer *renderer)

  La fonction vide le contenu graphique du renderer lié à l'écran de jeu.
- void update_screen (SDL_Renderer *renderer)

  La fonction met à jour l'écran avec le contenu du renderer.
- void pause (int time)

  La fonction met le programme en pause pendant un laps de temps.
- void clean_sdl (SDL_Renderer *renderer, SDL_Window *window)

  La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

4.7.1 Detailed Description

sur-couche de SDL2 pour simplifier son utilisation pour le projet

Author

Mathieu Constant

Version

0.2

Date

10 mars 2021

4.7.2 Function Documentation

4.7.2.1 apply_texture()

```
void apply_texture (
            SDL_Texture * texture,
            SDL_Renderer * renderer,
            int x,
            int y )
```

La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.

Parameters

texture	la texture que l'on va appliquer
renderer	le renderer qui va recevoir la texture
x	l'abscisse sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)
y	l'ordonnée sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)

4.7.2.2 clean_sdl()

```
void clean_sdl (
            SDL_Renderer * renderer,
            SDL_Window * window )
```

La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

Parameters

renderer	le renderer à nettoyer
window	la fenêtre à nettoyer

4.7.2.3 clean_texture()

```
void clean_texture (
            SDL_Texture * texture )
```

La fonction nettoie une texture en mémoire.

Parameters

texture	la texture à nettoyer

4.7.2.4 clear_renderer()

```
void clear_renderer (
            SDL_Renderer * renderer )
```

La fonction vide le contenu graphique du renderer lié à l'écran de jeu.

Parameters

renderer	le renderer de l'écran

4.7.2.5 init_sdl()

```
int init_sdl (
            SDL_Window ** window,
            SDL_Renderer ** renderer,
            int width,
            int height )
```

La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.

Parameters

window	la fenêtre du jeu
renderer	le renderer
width	largeur de l'écran de jeu
height	hauteur de l'écran de jeu

Returns

-1 en cas d'erreur, 0 sinon

4.7.2.6 load_image()

```
SDL_Texture* load_image (
            const char path[],
            SDL_Renderer * renderer )
```

La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.

Parameters

path	est le chemin du fichier image. Le fichier doit être obligatoirement du BMP.
renderer	le renderer

Returns

la surface SDL contenant l'image avec la couleur RGB (255,0,255) rendue transparente. Elle renvoie NULL si le chargement a échoué (ex. le fichier path n'existe pas)

4.7.2.7 pause()

```
void pause (
            int time )
```

La fonction met le programme en pause pendant un laps de temps.

Parameters

time	ce laps de temps en milliseconde

4.7.2.8 update_screen()

```
void update_screen (
            SDL_Renderer * renderer )
```

La fonction met à jour l'écran avec le contenu du renderer.

Parameters

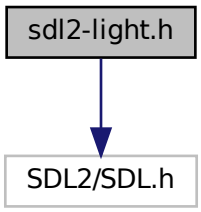
renderer	le renderer de l'écran

4.8 sdl2-light.h File Reference

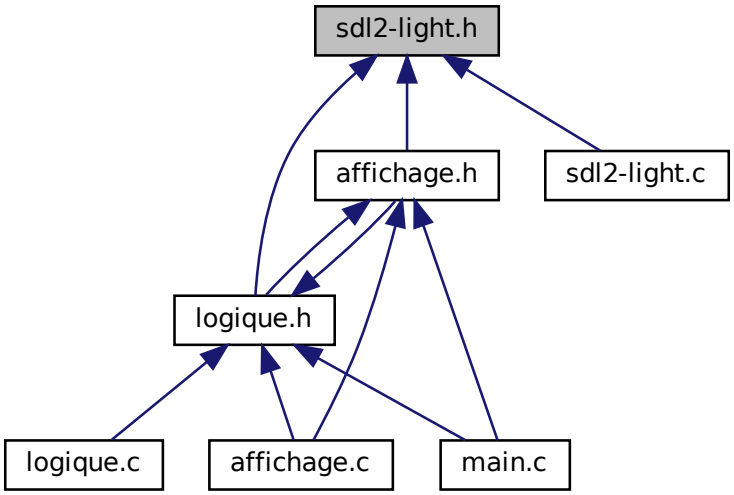
en-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet

```
#include <SDL2/SDL.h>
```

Include dependency graph for sdl2-light.h:



This graph shows which files directly or indirectly include this file:



Functions

- void clean_sdl (SDL_Renderer *renderer, SDL_Window *window)

  La fonction nettoie le renderer et la fenêtre du jeu en mémoire.
- SDL_Texture * load_image (const char path[ ], SDL_Renderer *renderer)

  La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.
- int init_sdl (SDL_Window **window, SDL_Renderer **renderer, int width, int height)

  La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.
- void clean_texture (SDL_Texture *texture)

  La fonction nettoie une texture en mémoire.

- void apply_texture (SDL_Texture ∗texture, SDL_Renderer ∗renderer, int x, int y)

  La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.
- void clear_renderer (SDL_Renderer ∗renderer)

  La fonction vide le contenu graphique du renderer lié à l'écran de jeu.
- void update_screen (SDL_Renderer ∗renderer)

  La fonction met à jour l'écran avec le contenu du renderer.
- void pause (int time)

  La fonction met le programme en pause pendant un laps de temps.

4.8.1 Detailed Description

en-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet

Author

Mathieu Constant

Version

0.2

Date

10 mars 2021

4.8.2 Function Documentation

4.8.2.1 apply_texture()

```
void apply_texture (
            SDL_Texture * texture,
            SDL_Renderer * renderer,
            int x,
            int y )
```

La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.

Parameters

texture	la texture que l'on va appliquer
renderer	le renderer qui va recevoir la texture
x	l'abscisse sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)
y	l'ordonnée sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)

4.8.2.2 clean_sdl()

```
void clean_sdl (
            SDL_Renderer * renderer,
            SDL_Window * window )
```

La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

Parameters

renderer	le renderer à nettoyer
window	la fenêtre à nettoyer

4.8.2.3 clean_texture()

```
void clean_texture (
            SDL_Texture * texture )
```

La fonction nettoie une texture en mémoire.

Parameters

texture	la texture à nettoyer

4.8.2.4 clear_renderer()

```
void clear_renderer (
            SDL_Renderer * renderer )
```

La fonction vide le contenu graphique du renderer lié à l'écran de jeu.

Parameters

renderer	le renderer de l'écran

4.8.2.5 init_sdl()

```
int init_sdl (
            SDL_Window ** window,
```

```
            SDL_Renderer ** renderer,
            int width,
            int height )
```

La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.

Parameters

window	la fenêtre du jeu
renderer	le renderer
width	largeur de l'écran de jeu
height	hauteur de l'écran de jeu

Returns

-1 en cas d'erreur, 0 sinon

4.8.2.6 load_image()

```
SDL_Texture* load_image (
            const char path[],
            SDL_Renderer * renderer )
```

La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.

Parameters

path	est le chemin du fichier image. Le fichier doit être obligatoirement du BMP.
renderer	le renderer

Returns

la surface SDL contenant l'image avec la couleur RGB (255,0,255) rendue transparente. Elle renvoie NULL si le chargement a échoué (ex. le fichier path n'existe pas)

4.8.2.7 pause()

```
void pause (
            int time )
```

La fonction met le programme en pause pendant un laps de temps.

Parameters

time	ce laps de temps en milliseconde

4.8.2.8 update_screen()

```
void update_screen (
            SDL_Renderer * renderer )
```

La fonction met à jour l'écran avec le contenu du renderer.

Parameters

renderer	le renderer de l'écran

Index

affichage.c, 11
    apply_background, 12
    apply_sprite, 12
    apply_texts, 13
    apply_walls, 13
    clean_ressources, 13
    init_textures, 14
    refresh_graphics, 14
affichage.h, 14
    apply_background, 16
    apply_sprite, 17
    apply_texts, 17
    apply_walls, 17
    clean_ressources, 18
    init_textures, 18
    refresh_graphics, 18
apply_background
    affichage.c, 12
    affichage.h, 16
apply_sprite
    affichage.c, 12
    affichage.h, 17
apply_texts
    affichage.c, 13
    affichage.h, 17
apply_texture
    sdl2-light.c, 36
    sdl2-light.h, 40
apply_walls
    affichage.c, 13
    affichage.h, 17

background
    ressources_s, 5

clean
    main.c, 33
clean_ressources
    affichage.c, 13
    affichage.h, 18
clean_sdl
    sdl2-light.c, 36
    sdl2-light.h, 41
clean_texture
    sdl2-light.c, 36
    sdl2-light.h, 41
clear_renderer
    sdl2-light.c, 37
    sdl2-light.h, 41
constantes.h, 19

font
    ressources_s, 5

gameover
    world_s, 8

h
    sprite_s, 7
handle_events
    main.c, 33
handle_sprites_collision
    logique.c, 21
    logique.h, 28

init
    main.c, 34
init_data
    logique.c, 21
    logique.h, 28
init_sdl
    sdl2-light.c, 37
    sdl2-light.h, 41
init_sprite
    logique.c, 23
    logique.h, 29
init_textures
    affichage.c, 14
    affichage.h, 18
init_walls
    logique.c, 23
    logique.h, 29
is_finished
    logique.c, 23
    logique.h, 29

ligne_arrivee
    ressources_s, 6
    world_s, 8
load_image
    sdl2-light.c, 37
    sdl2-light.h, 42
logique.c, 20
    handle_sprites_collision, 21
    init_data, 21
    init_sprite, 23
    init_walls, 23
    is_finished, 23
    shoot_missile, 24
    sprites_collide, 24
    update_data, 24

update_finishline, 25
update_missiles, 25
update_time, 25
update_walls, 25
logique.h, 26
handle_sprites_collision, 28
init_data, 28
init_sprite, 29
init_walls, 29
is_finished, 29
shoot_missile, 30
sprites_collide, 30
update_data, 30
update_finishline, 31
update_missiles, 31
update_time, 31
update_walls, 32

main.c, 32
clean, 33
handle_events, 33
init, 34
meteorite
ressources_s, 6
missile
ressources_s, 6
missiles
world_s, 9

pause
sdl2-light.c, 38
sdl2-light.h, 42

refresh_graphics
affichage.c, 14
affichage.h, 18
ressources_s, 5
background, 5
font, 5
ligne_arrivee, 6
meteorite, 6
missile, 6
vaisseau, 6

sdl2-light.c, 34
apply_texture, 36
clean_sdl, 36
clean_texture, 36
clear_renderer, 37
init_sdl, 37
load_image, 37
pause, 38
update_screen, 38
sdl2-light.h, 38
apply_texture, 40
clean_sdl, 41
clean_texture, 41
clear_renderer, 41
init_sdl, 41
load_image, 42
pause, 42
update_screen, 43
shoot_missile
logique.c, 24
logique.h, 30
sprite_s, 6
h, 7
w, 7
x, 7
y, 7
sprites_collide
logique.c, 24
logique.h, 30

time
world_s, 9

update_data
logique.c, 24
logique.h, 30
update_finishline
logique.c, 25
logique.h, 31
update_missiles
logique.c, 25
logique.h, 31
update_screen
sdl2-light.c, 38
sdl2-light.h, 43
update_time
logique.c, 25
logique.h, 31
update_walls
logique.c, 25
logique.h, 32

vaisseau
ressources_s, 6
world_s, 9
vy
world_s, 9

w
sprite_s, 7
walls
world_s, 9
win
world_s, 9
world_s, 8
gameover, 8
ligne_arrivee, 8
missiles, 9
time, 9
vaisseau, 9
vy, 9
walls, 9
win, 9

x

sprite_s, 7

y
sprite_s, 7